

AGENTIC RAG PIPELINE

An Autonomous, Multi-Hop Retrieval-Augmented Generation System

FastAPI • Groq LLM • ChromaDB • React + Vite + Tailwind

Project Documentation & Technical Report

Author: Uzair Ali

Repository: github.com/uzairalimay22-cmyk/Agentic-RAG-Full-Project

June 2026



Contents

1. Executive Summary
2. System Architecture
3. Technology Stack
4. Key Features
5. The Agentic Pipeline & Event Trace
6. Project Structure
7. REST API Reference
8. Configuration Reference
9. Setup & Installation Guide
10. Using the Application
11. Conclusion & Future Work

This document describes the design, architecture, and usage of the Agentic RAG Pipeline — a Retrieval-Augmented Generation system built around an autonomous LLM agent that decides for itself when and how to search a private document knowledge base, with a live, streaming visualization of every step in the pipeline.

1. Executive Summary

The **Agentic RAG Pipeline** is a full-stack Retrieval-Augmented Generation (RAG) application that goes beyond the conventional “retrieve-then-generate” pattern. Instead of always running a fixed retrieval step before generation, the system gives a large language model (LLM) **agency**: the model itself decides whether a question requires searching the user's documents, issues one or more refined searches (multi-hop retrieval), and only then produces a final answer.

The backend is a Python **FastAPI** service that exposes document ingestion, knowledge-base management, and a streaming chat endpoint. Documents (PDF, DOCX, TXT, MD) are chunked and embedded locally using Chroma's bundled ONNX MiniLM model, then stored in a persistent **ChromaDB** vector database — no external embedding API or key is required. Reasoning is delegated to **Groq**, a free, OpenAI-compatible LLM API with very fast inference and native tool (function) calling, running the **llama-3.3-70b-versatile** model by default.

The frontend is a **React 19 + Vite + Tailwind CSS 4** single-page application. Its centerpiece is a **live pipeline visualizer** that consumes a stream of newline-delimited JSON (NDJSON) events from the backend and renders, in real time, exactly which stage of the pipeline is active — agent reasoning, tool selection, knowledge-base search, tool results, and final answer generation — along with which model and component is responsible for each step.

This makes the system useful both as a practical document Q&A tool and as an educational demonstration of how agentic RAG systems work internally.

2. System Architecture

The diagram below shows the end-to-end data flow, from document ingestion on the left/top path to the live pipeline trace consumed by the frontend.



Figure 1. Data flow through the ingestion pipeline (top), the agentic query loop (middle), and the streamed pipeline trace consumed by the UI (bottom).

Design rationale

- **Agentic, not fixed, RAG** — the agent (default `llama-3.3-70b-versatile` on Groq) decides for itself whether to call `search_knowledge_base`, can issue multiple refined searches (multi-hop retrieval), and can also call `list_knowledge_base_documents`. This is a real agent loop, not a fixed retrieve-then-generate pipeline.
- **Free LLM provider** — Groq offers a generous free tier with fast inference and OpenAI-compatible tool calling, requiring only a free API key (no credit card).
- **Local embeddings** — Chroma's bundled ONNX MiniLM model (`all-MiniLM-L6-v2`) runs locally via `onnxruntime`, so no separate embeddings provider or API key is needed.
- **ChromaDB** — a local, file-based, persistent vector database requiring no external service or account.
- **Streamed pipeline trace** — the backend streams NDJSON events (`pipeline_step`, `tool_call`, `tool_result`, `thinking`, `token`, `final`, `done`, `error`) over `/api/chat`. The frontend turns these into a live diagram and step-by-step log.

3. Technology Stack

Backend

Component	Technology	Purpose
Web framework	FastAPI + Uvicorn	REST API and NDJSON streaming endpoint
LLM provider	Groq SDK (groq>=1.4.0)	Agentic reasoning & tool calling (llama-3.3-70b-versatile)
Vector database	ChromaDB 0.5.23	Persistent local vector store for document chunks
Embeddings	ONNX MiniLM (all-MiniLM-L6-v2)	Local, free text embeddings via onnxruntime 1.18.1
Document parsing	pypdf, python-docx	Extract text from PDF / DOCX files
Chunking	langchain-text-splitters	RecursiveCharacterTextSplitter for overlap-aware chunks
Settings	pydantic-settings	Typed configuration loaded from backend/.env

Frontend

Component	Technology	Purpose
UI library	React 19	Component-based single-page application
Build tool	Vite 8	Dev server & production bundling
Styling	Tailwind CSS 4	Utility-first styling, dark themed UI
Markdown	react-markdown	Render assistant answers with formatting
Icons	lucide-react	Icon set used across the UI
Language	TypeScript	Type-safe components and API client

4. Key Features

Feature	Description
Agentic multi-hop retrieval	The LLM decides whether to search at all, and may issue multiple, progressively refined searches before answering — useful for multi-part questions.
Live pipeline visualizer	A real-time diagram on the right side of the UI highlights the active stage (user query, agent reasoning, tool call, knowledge base, response) and shows which model/component is running.
Step-by-step trace log	Every event (tool selected, tool call, tool result, final answer, errors) is logged in a scrollable panel so the entire reasoning process is auditable.
Document management	Upload PDF/DOCX/TXT/MD files via the sidebar; documents are chunked, embedded, and stored automatically. List and delete documents (and their chunks) at any time.
Source-cited answers	The system prompt instructs the agent to cite the source filename for any information drawn from the knowledge base.
Zero-cost local embeddings	Embeddings run fully locally via ONNX — only the LLM call requires an external (free) API.
Streaming responses	Both reasoning tokens and the final answer are streamed to the client over NDJSON for a responsive chat experience.

5. The Agentic Pipeline & Event Trace

The core of the system is `run_agent()` in `backend/app/agent/orchestrator.py`. It runs an iterative loop (up to **6 iterations**) against the Groq chat completions API with two tools available to the model:

Tool	Description
<code>search_knowledge_base</code>	Semantic search over the user's uploaded documents. May be called multiple times with different queries.
<code>list_knowledge_base_documents</code>	Lists all documents currently stored, including chunk counts — used to check what information is available.

Agent loop, step by step

- 1 The system prompt instructs the agent on when to search, how to cite sources, and how to handle questions unrelated to the documents.
- 2 The user's message (plus prior conversation history) is sent to the Groq model as a streaming chat-completion request, with both tools registered.
- 3 If the model responds with `tool_calls`, each call is executed locally (against ChromaDB), the result is appended to the conversation as a `tool` message, and the loop repeats — enabling multi-hop retrieval.
- 4 If the model responds with plain content and no further tool calls, that content is streamed back as the final answer and the loop ends.
- 5 If neither happens within 6 iterations, an error event is emitted.

Pipeline trace event types

Every step of the loop yields a small JSON event over `/api/chat`, allowing the frontend to render a live trace:

Event type	Meaning
<code>pipeline_step</code>	A named stage change, e.g. query received, agent reasoning (with active model), tool selected, or answer rendered.
<code>tool_call</code>	The agent is invoking a tool, with its name and parsed arguments.
<code>tool_result</code>	The result returned from executing that tool (e.g. retrieved chunks).
<code>thinking</code>	Optional model reasoning tokens, if provided by the API.
<code>token</code>	A streamed fragment of the assistant's response text.
<code>final</code>	The complete final answer text.
<code>done</code>	Marks the end of the stream (success or after an error).
<code>error</code>	A fatal error (e.g. missing API key, Groq API error, iteration limit reached).

6. Project Structure

```
Agentic-RAG-Full-Project/
  backend/
    app/
      main.py          # FastAPI app & routes
      config.py        # Settings (.env driven)
      models.py        # Pydantic schemas
    rag/
      ingestion.py     # Document loading + chunking
      vectorstore.py   # ChromaDB wrapper (local embeddings)
    agent/
      tools.py         # Tool definitions for the agent
      orchestrator.py  # Agentic loop + pipeline trace events
  requirements.txt
  .env.example
  run.py
  frontend/
    src/
      components/      # Chat, DocumentSidebar, PipelineVisualizer, ...
      api.ts           # Backend API client (incl. NDJSON streaming)
      types.ts         # Shared TypeScript types
```

7. REST API Reference

Endpoint	Method	Description
/api/health	GET	Health check.
/api/config	GET	Active model / embedding / vector-store info for the UI (incl. whether the Groq key is correct).
/api/upload	POST	Upload + ingest a document (multipart form, field 'file').
/api/documents	GET	List ingested documents with chunk counts and sizes.
/api/documents/{source}	DELETE	Delete a document and all of its chunks.
/api/chat	POST	Body: { message, history }. Returns a streamed NDJSON pipeline trace + final answer.

Example: streaming chat request

POST /api/chat
Content-Type: application/json

```
{
  "message": "What does the report say about Q3 revenue?",
  "history": []
}
```

Response (application/x-ndjson, one JSON object per line):

```
{ "type": "pipeline_step", "step": "query_received", ... }
{ "type": "pipeline_step", "step": "agent_call", "model": "llama-3.3-70b-versatile" }
{ "type": "tool_call", "detail": { "name": "search_knowledge_base", ... } }
{ "type": "tool_result", "detail": { "results": [...] } }
{ "type": "token", "text": "Based on the report (source: q3.pdf), ..." }
{ "type": "final", "text": "..." }
{ "type": "done" }
```

8. Configuration Reference

All configuration is supplied via backend/.env (see backend/.env.example):

Variable	Default	Description
GROQ_API_KEY	(required)	Free Groq API key from console.groq.com/keys.
GROQ_MODEL	llama-3.3-70b-versatile	Model used by the agent.
CORS_ORIGINS	http://localhost:5173, http://127.0.0.1:5173	Comma-separated list of allowed frontend origins.
CHUNK_SIZE / CHUNK_OVERLAP	1000 / 200	Text-splitter chunk size and overlap (characters).
DEFAULT_TOP_K	5	Default number of chunks retrieved per search.
COLLECTION_NAME	knowledge_base	Name of the ChromaDB collection.

Embeddings always use Chroma's bundled local ONNX MiniLM model (all-MiniLM-L6-v2) — no extra variable or key is needed for embeddings.

9. Setup & Installation Guide

1. Backend

```
cd backend
python -m venv venv
./venv/Scripts/Activate.ps1 # or venv\Scripts\activate on cmd
pip install -r requirements.txt
```

```
cp .env.example .env
# edit .env and set GROQ_API_KEY=gsk_...
# (free key: https://console.groq.com/keys)
```

```
python run.py
```

The API runs at `http://localhost:8000`. The first request that touches the vector store automatically downloads the local embedding model (~80 MB).

2. Frontend

```
cd frontend
npm install
npm run dev
```

The UI runs at `http://localhost:5173` and talks to the backend at `http://localhost:8000`.

10. Using the Application

- 1 Open the UI and upload one or more documents (PDF, DOCX, TXT, MD) via the sidebar — they are chunked, embedded locally, and stored in ChromaDB.
- 2 Ask a question in the chat box. Watch the pipeline visualizer light up each stage as it happens: agent reasoning with the active Groq model, tool calls to `search_knowledge_base`, chunks retrieved from ChromaDB, and final answer generation.
- 3 For multi-part questions, the agent decides on its own whether and how many times to search — you'll often see several `search_knowledge_base` calls before the final answer (multi-hop retrieval).
- 4 Review the step-by-step log on the right for a full audit trail of every reasoning step, tool call, and tool result in the conversation.
- 5 Manage the knowledge base from the sidebar: view document counts and sizes, or delete a document (and its chunks) when it's no longer needed.

11. Conclusion & Future Work

The Agentic RAG Pipeline demonstrates a practical, low-cost (free LLM tier, local embeddings, local vector store) implementation of an agent-driven RAG system with full transparency into its reasoning process via a live pipeline trace. It serves both as a usable document-Q&A tool and as a teaching aid for how modern agentic RAG architectures work.

Potential future enhancements

- Support for additional LLM providers (e.g. OpenAI, Anthropic) via a pluggable provider layer.
- Persisted chat history / multi-conversation support with a lightweight database.
- Re-ranking of retrieved chunks before they are passed to the agent.
- Inline citation highlighting in the UI, linking answers back to source chunks.
- Authentication and per-user knowledge bases for multi-tenant deployments.
- Automated evaluation harness for retrieval quality and answer faithfulness.